

```

1  # Note 02: Finding the Best Fitted Line (RMSE Method)
2  # =====
3
4  # Import Libraries
5  import pandas as pd
6  import matplotlib.pyplot as plt
7  import numpy as np
8
9  # 01. Load Dataset
10 # =====
11
12 df = pd.read_csv('test_data.csv')
13
14 # 02. Define X and Y Variables
15 # =====
16
17 X = df['temperature'].tolist()
18 Y = df['volt'].tolist()
19 print("\n01. Type of X : ", type(X).__name__)
20 print("02. Type of Y : ", type(Y).__name__)
21
22 # 03. Draw a Scatter Plot to Identify the Relationship
23 # =====
24
25 plt.figure()
26 plt.scatter(X, Y, marker='o', label='Original Data', c='red', s=20)
27 plt.plot(X, Y, label='Original Data', c='blue')
28 plt.title('Temperature vs Voltage (Original Data)')
29 plt.xlabel('Temperature')
30 plt.ylabel('Voltage')
31 plt.grid(True, alpha = 0.3)
32 plt.legend()
33 plt.show()
34
35 # Print Coefficients of Order 1
36 coefficients = np.polyfit(X, Y, 1)
37 print("03. Slope of the Polynomial [Order 01] : ", coefficients[0])
38 print("04. Intercept of the Polynomial [Order 01] : ", coefficients[1])
39
40 # 04. Draw Elbow Plot to Find Best RMSE Value
41 # =====
42
43 # Function to Draw Elbow Plot
44 def draw_elbow_plot(start, end, rmse_values, orders, x, y):
45
46     for order in range(start, end): # start <= order < end
47         # Fit polynomial
48         coefficients = np.polyfit(x, y, order)
49         # Create polynomial function
50         polynomial = np.poly1d(coefficients)
51         # Predicted y values
52         y_pred = polynomial(x)
53         # Calculate RMSE
54         rmse = np.sqrt(np.mean((y - y_pred)**2))
55         # Store values
56         orders.append(order)
57         rmse_values.append(rmse)
58
59     # Plot the Order vs RMSE Graph
60     plt.figure()
61     plt.scatter(orders, rmse_values, marker='o', s = 20, c='red')
62     plt.plot(orders, rmse_values, c='blue')
63     plt.xlabel('Order')
64     plt.ylabel('RMSE')
65     plt.title('Order vs RMSE')
66     plt.xticks()
67     plt.grid(True, alpha = 0.3)
68     plt.plot()
69     plt.show()
70
71 # Call to the Function
72 rmse_values = []

```

```

73 orders = []
74 draw_elbow_plot(1, 9, rmse_values, orders, X, Y)
75 rmse_values = [float(x) for x in rmse_values]
76 print("05. RMSE List :", rmse_values)
77 print("06. Order List :", orders)
78 print("07. Best Order : 3") # From Looking at Elbow Method
79
80 # Function to Plot Smooth Curve for Best Fitted Line
81 def smooth_curve(x, y, best_order, intervals):
82     coefficients = np.polyfit(x, y, best_order)
83     f = np.polyld(coefficients)
84     x_new = np.linspace(min(x), max(x), intervals)
85     y_new = f(x_new)
86     plt.figure()
87     plt.scatter(x, y, marker='o', label='Original Data', c='red', s=20)
88     plt.plot(x_new, y_new, c='blue', label='Smooth Curve')
89     plt.title('Best Fitted Line')
90     plt.xlabel('X')
91     plt.ylabel('Y')
92     plt.grid(True, alpha = 0.3)
93     plt.legend()
94     plt.show()
95
96     # Find Y Value For Given X
97     print("Y value when X = 33 : ", f(33))
98
99 # Call a Function to Plot Smooth Curve
100 smooth_curve(X, Y, 3, 10000)
101
102 # 05. Find X Value For Given Y [Inverse Function]
103 # =====
104
105 # Define X and Y by Changing the Values
106 X_inv = Y
107 Y_inv = X
108
109 # Define Two Empty Lists to Store RMSE and Orders
110 rmse_values_inv = []
111 orders_inv = []
112
113 # Call Two Functions to Get Best Fitted Line For Inverse Function
114 draw_elbow_plot(1, 9, rmse_values_inv, orders_inv, X_inv, Y_inv)
115 smooth_curve(X_inv, Y_inv, 3, 10000)

```